TECHNICAL SEMINAR REPORT

RasoAI: INGREDIENT-BASED INDIAN RECIPE RECOMMENDATION SYSTEM

Group ID: **F12**

FOR THE DEGREE OF

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

Shardul Kamde (PRN No.: SOE24201010462)

Samvat Jain (PRN No.: SOE24201010443)

Abhishek Ghatti (PRN No.: SOE24201010154)

Aryan Singh (PRN No.: SOE24201010045)

UNDER THE GUIDANCE OF

Dr. Amruta Hingmire

PCET's Pimpri Chinchwad University

Learn | Grow | Achieve

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

PIMPRI CHINCHWAD UNIVERSITY

49/50, Plot No. 44, Mohitewadi Rd, Mohitewadi,

Maharashtra 412106

YEAR 2026-27

PCET's
Pimpri
Chinchwad
University

Learn | Grow | Achieve

CERTIFICATE

This is to certify that the seminar report entitled

RasoAI: Ingredient-Based Indian Recipe Recommendation System

Submitted by

Shardul Kamde (PRN No.: SOE24201010462), Samvat Jain (PRN No.: SOE24201010443), Abhishek Ghatti (PRN No.: SOE24201010154), and Aryan Singh (PRN No.: SOE24201010045)

are bona fide students of this institute and the work has been carried out by them under the supervision of Dr. Amruta Hingmire and is approved for the partial fulfillment of the requirement of PIMPRI CHINCHWAD UNIVERSITY, for the award of Third Year of Bachelor of Technology (CSE) A.Y.-2026-27.

Dr. Amruta Hingmire	Ms. Rajashree Landage	Dr. Pallavi Ahire
Guide	Technical Seminar	Head of Department
Department of CSE	Coordinator	Department of CSE
	Department of CSE	

Place: _____ Date: _____

ACKNOWLEDGEMENT

We express our sincere thanks to our project guide for valuable suggestions, encouragement, and support throughout the development of RasoAI. The guidance helped us shape the project scope, refine the recipe-matching workflow, and complete this seminar report.

We also thank our Technical Seminar Coordinator, Ms. Rajashree Landage, Academic Coordinator Dr. Sandip Sonawane, Head of the Department Dr. Pallavi Ahire, and Vice Chancellor Dr. Santosh S. Sonavane for their guidance and for providing the academic environment in which this work could be completed.

Finally, we thank our department and our families for their encouragement and support.

Shardul Kamde
Samvat Jain
Abhishek Ghatti
Aryan Singh
Students, Department of CSE

ABSTRACT

RasoAI is a local recipe-finding application that recommends an Indian recipe from ingredients supplied by a user. The project addresses the practical problem of finding a usable recipe when a person has ingredients available but does not know which dish to prepare. Its recipe collection is stored in an Excel workbook containing recipe names, ingredients, instructions, cuisine, course, diet, preparation and cooking times, servings, and source URLs.

The system combines a Streamlit web interface with a FastAPI backend. The backend loads the workbook with pandas and OpenPyXL, normalizes ingredient text, and uses RapidFuzz string similarity to accommodate small spelling variations. A recipe is considered only when at least half of the entered ingredients match; eligible recipes are ranked by ingredient coverage and a keyword-overlap score. The application returns recipe details and dataset instructions, which the interface presents as numbered, gradually revealed steps. The implementation is a deterministic retrieval system: it does not generate recipes with a large language model. Unit tests cover exact ingredient matching, spelling variation, ranking, and insufficient ingredient coverage.

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Motivation . . . 1
- 1.3 Problem Statement and Objectives . . . 1
- 1.4 Scope . . . 2
- 1.5 Methodologies of Problem Solving . . . 2

2 Literature Survey **3**

3 Hardware/Software Requirements Specification **4**
- 3.1 System Requirements . . . 4
  - 3.1.1 Database Requirements . . . 4
  - 3.1.2 Software Requirements (Platform Choice) . . . 4
  - 3.1.3 Hardware Requirements . . . 4
- 3.2 Analysis Models: SDLC Model to be Applied . . . 5

4 System Design **6**
- 4.1 System Architecture . . . 6

5 Implementation **7**
- 5.1 Overview of Technical Seminar–II Modules . . . 7
- 5.2 Tools and Technologies Used . . . 7
- 5.3 Algorithm Details . . . 8
  - 5.3.1 Ingredient Normalization and Candidate Eligibility . . . 8
  - 5.3.2 Candidate Scoring and Ranking . . . 8

6 Results **9**
- 6.1 Result Analysis and Validations . . . 9
- 6.2 Representative User Workflow . . . 10

7 Conclusions and Future Scope **11**
- 7.1 Conclusion . . . 11
- 7.2 Future Scope . . . 11

A API and Local Execution **12**
- A.1 Project Limitations . . . 12

1. INTRODUCTION

1.1 Overview

RasoAI is a local, ingredient-based recipe recommendation system. A user selects ingredients from searchable choices derived from the recipe workbook or enters custom ingredients. The Streamlit frontend sends the selection to a FastAPI service, which searches the recipes loaded from the Excel dataset and returns the best eligible match.

The application has three main responsibilities: presenting and validating user input, retrieving a suitable recipe, and displaying the recipe's source data. The interface shows recipe metadata, the dataset ingredient list, the original recipe URL, and the instructions. Instructions are separated into readable numbered steps; the recipe content itself is sourced from the workbook rather than generated by an LLM.

1.2 Motivation

Recipe discovery is often organized around a dish name or a single search phrase. That approach is less helpful when the user starts with ingredients already on hand. RasoAI reverses the search process: it accepts the available ingredients first and ranks recipes according to how well they match that list.

This project was selected to apply data processing, text normalization, approximate string matching, API design, and user-interface development to a familiar real-world task. Its local dataset and local services also make the demonstrated workflow usable without calling a hosted generative-AI service.

1.3 Problem Statement and Objectives

Problem statement: Given a list of ingredients supplied by a user, identify and present a relevant recipe from a structured Indian-food dataset, while tolerating limited spelling variation and avoiding unsupported recipe generation.

Objectives:

1. Load and validate recipe records from the Excel workbook.
2. Provide searchable dataset ingredients and custom ingredient entry.
3. Match and rank recipes using approximate keyword comparison and minimum coverage.
4. Return recipe details through the API and present them with numbered instructions.
5. Validate matching behavior with automated unit tests.

1.4 Scope

The current system is a single-user local application backed by a static Excel workbook. It searches the records in that workbook and returns one best match. Users can select from dataset-derived suggestions or provide comma-separated custom ingredients. The API exposes a recipe-search endpoint and an endpoint for ingredient suggestions.

The project does not include user accounts, personalization, ratings, pantry persistence, nutritional analysis, a relational database, live web scraping, or automatic recipe generation. The dataset determines the available recipes and source URLs; updating or expanding the dataset requires updating the workbook. The matching score is a lexical ranking aid, not a measure of recipe quality, dietary suitability, or nutritional safety.

1.5 Methodologies of Problem Solving

1. **Data preparation:** inspect required workbook fields, load recipes, normalize ingredients, and derive keywords.
2. **Matching:** compare query and recipe keywords with RapidFuzz; enforce 50% coverage and rank by coverage and overlap.
3. **Integration:** provide ingredient suggestions and recipe search through FastAPI, with selection and results in Streamlit.
4. **Validation:** test exact and approximate matches, ranking, and insufficient ingredient coverage.

2. LITERATURE SURVEY

Recommender systems select or rank items according to signals that connect a user or query to candidate items. The general design literature distinguishes content-based approaches, which compare item attributes, from collaborative approaches, which use interactions among users and items [1]. RasoAI is content-based: recipe ingredient keywords are compared with the ingredients in the query. It does not collect user ratings or interaction history and therefore does not implement collaborative filtering.

Information retrieval systems commonly normalize query and document terms before comparing them. This reduces irrelevant surface variation and enables a term-based comparison [2]. RasoAI applies a lightweight version of this idea: punctuation and quantities are removed, common measurement and preparation words are excluded, and the remaining ingredient keywords are compared. This design is appropriate for a small local recipe-search workflow, but normalization can also lose context; multiword ingredient phrases are therefore treated as the set of meaningful words extracted from that phrase.

Exact string comparison is brittle when a user makes a small spelling error. Approximate string matching addresses this issue by measuring character-level similarity; RapidFuzz provides the ratio-based comparison used by this implementation [5]. RasoAI accepts a keyword match at a ratio of at least 90 and requires at least half of the entered ingredient items to match a candidate recipe. The threshold is an implementation choice and should be evaluated on additional labelled queries before being treated as an optimal value.

FastAPI provides the HTTP interface between the user-facing application and the recipe-search logic [3]. Streamlit supplies the interactive ingredient-selection interface and result display [4]. Pandas and OpenPyXL provide workbook ingestion and spreadsheet support [6, 7]. In contrast to a generative chatbot, this system returns records from a fixed dataset; this makes the provenance of the recipe instructions explicit and avoids presenting newly generated directions as source material.

3. HARDWARE/SOFTWARE REQUIREMENTS SPECIFICATION

3.1 System Requirements

3.1.1 Database Requirements

RasoAI does not use a database server. Its data store is the supplied Excel workbook, `backend/data/IndianFoodDatasetXLS.xlsx`. At startup, pandas reads the workbook and the backend constructs an in-memory list of recipe records. The required columns include recipe identifier, name, ingredients, instructions, cuisine, course, diet, preparation time, cooking time, total time, servings, and URL. Keyword sets are derived from the ingredient text. The application is read-only: it does not write user selections or recipe changes to the workbook.

3.1.2 Software Requirements (Platform Choice)

- **Language and runtime:** Python 3.10 or later.
- **Backend:** FastAPI, Pydantic, and Uvicorn.
- **Frontend:** Streamlit and Requests.
- **Data access:** pandas and OpenPyXL for reading the Excel workbook.
- **Approximate matching:** RapidFuzz.
- **Testing:** Python `unittest`.
- **Version control:** Git and GitHub.

The backend and frontend dependencies are listed separately in their respective `requirements.txt` files. The backend is started from the `backend` directory so its relative dataset path resolves correctly; the Streamlit application is started from the `frontend` directory.

3.1.3 Hardware Requirements

The application is intended for a general-purpose computer capable of running Python and a modern web browser. It requires local storage for the Excel workbook,

source files, Python packages, and optional background video. No GPU or specialized hardware is required. The exact memory and storage requirements depend on the Python environment and local dataset; a fixed hardware benchmark was not part of this project.

3.2 Analysis Models: SDLC Model to be Applied

The project was developed using an iterative, incremental workflow. The dataset loader and matching logic were implemented first, followed by API endpoints, the interactive interface, and result presentation. Each increment could be run locally and checked before further interface or documentation work was added. Unit tests were used to validate important matching rules. This workflow suited the project because the user experience and ranking behavior could be refined as the data and end-to-end application were exercised.

4. SYSTEM DESIGN

4.1 System Architecture

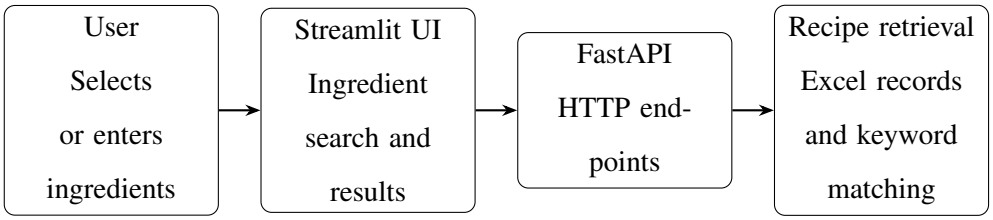


Figure 4.1. RasoAI high-level system architecture

The Excel workbook is loaded when the backend starts. The Streamlit frontend requests dataset-derived ingredient choices from `GET /ingredients`. On submission, it sends the chosen ingredients as JSON to `POST /chat`. FastAPI passes the query to the recipe retriever, which compares the query with the in-memory recipe keywords and returns either a best match or a no-match response. The interface displays the recipe metadata, source link, ingredients, and instructions.

This separation keeps the presentation layer independent of the matching implementation. The API can also be inspected through the automatically generated FastAPI documentation at `/docs`.

5. IMPLEMENTATION

5.1 Overview of Technical Seminar–II Modules

- **Dataset loader (`dataset_loader.py`):** reads the Excel workbook, verifies required columns, and constructs recipe records with normalized ingredient keywords.
- **Text utilities (`utils.py`):** lowercases ingredient text, removes quantities and punctuation, and filters measurement or preparation terms.
- **Recipe retriever (`recipe_retriever.py`):** compares query keywords with each recipe, enforces minimum ingredient coverage, scores eligible candidates, and returns the top match.
- **FastAPI service (`main.py`):** loads data at startup and implements `GET /`, `GET /ingredients`, and `POST /chat`.
- **Streamlit interface (`web_ui.py`):** provides searchable dataset ingredient buttons, custom text entry, recipe search, result details, source link, and a step-by-step instruction display.
- **Automated tests:** verify exact ingredient matching, fuzzy spelling tolerance, ranking behavior, and the minimum coverage rule.

5.2 Tools and Technologies Used

Python provides the application runtime. FastAPI and Pydantic define and validate the JSON API, while Uvicorn serves the backend. The Streamlit frontend renders the interactive application and uses Requests to call the local API. Pandas reads the tabular Excel data, with OpenPyXL as the spreadsheet engine. RapidFuzz performs approximate comparisons between normalized words. Git and GitHub provide version history and remote source-code hosting.

5.3 Algorithm Details

5.3.1 Ingredient Normalization and Candidate Eligibility

The text utility converts each ingredient string to lowercase, removes parenthetical text, digits, punctuation, and repeated whitespace, then filters stop words and short tokens. For each recipe, the loader stores the resulting keywords. At query time, the retriever checks each submitted ingredient against the recipe keyword set. A submitted ingredient is considered matched when every keyword extracted from that ingredient has a sufficiently similar recipe keyword. RapidFuzz ratio scores are compared with a threshold of 90.

```
for each recipe:
    matched = matching query ingredients
    coverage = 100 * matched / query size
    if coverage >= 50: keep recipe
```

5.3.2 Candidate Scoring and Ranking

Eligible recipes are ranked first by ingredient coverage. When coverage is equal, the retriever uses a Dice-style keyword-overlap score:

$$\text{score}(U, R) = \frac{2\,|M(U, R)|}{|U| + |R|} \times 100,$$

where U is the set of query keywords, R is the set of recipe keywords, and $M(U, R)$ is the number of query keywords that match a recipe keyword under the fuzzy threshold. The score is a lexical similarity indicator; it is not a probability that the recipe will be suitable or successful.

```
best = none; best_rank = (0, 0)
for each eligible recipe:
    rank = (coverage, keyword_overlap)
    if rank > best_rank: update best
return best, or no_match
```

6. RESULTS

6.1 Result Analysis and Validations

The workbook contains 6,871 recipe rows and the columns used by the application for ingredient matching and result display. The backend returns dataset-derived ingredient suggestions and recipe results through its API. The UI was exercised with selected ingredients and custom ingredient input, and displays the best-match metadata, source URL, dataset ingredients, and numbered instructions.

The automated test suite contains four focused tests. Table 6.1 summarizes their intended checks.

Table 6.1: Recipe retrieval validation cases

Test case	**Expected behavior**
Exact ingredient preference	Select the recipe containing the exact requested ingredient rather than a partial-word candidate.
Query-based scoring	Prefer the recipe that best covers the supplied ingredients, not simply the recipe with more ingredient words.
Minimum coverage	Return no recipe when fewer than half of the submitted ingredients match.
Spelling variation	Match a small spelling variation when its fuzzy similarity passes the configured threshold.

All four recipe-retriever unit tests pass in the development environment. The threshold and score are heuristic design choices; a larger labelled test set and user evaluation would be needed to quantify recommendation quality beyond these functional tests.

6.2 Representative User Workflow

1. Open the Streamlit application after starting the FastAPI backend.
2. Search for an ingredient or select one of the popular dataset-derived ingredient buttons.
3. Add optional comma-separated ingredients that are not in the suggestions.
4. Select **Generate recipe**.
5. Review the best recipe match, its cuisine/course/time metadata, the source link, and the dataset ingredients.
6. Read the instructions as numbered steps, which are revealed one line at a time.

The architecture diagram in Figure 4.1 documents the implemented data flow. A separate application screenshot is not included in this source file.

7. CONCLUSIONS AND FUTURE SCOPE

7.1 Conclusion

RasoAI demonstrates a local content-based recipe retrieval workflow built from a spreadsheet dataset. It connects a searchable ingredient-selection interface to a FastAPI service, normalizes ingredient text, tolerates small spelling variations, applies a minimum coverage threshold, and returns a ranked recipe record. Recipe instructions and metadata are sourced from the workbook rather than generated by a language model.

The implementation meets its core functional objectives: the dataset is loaded and validated, ingredient choices can be searched and selected, recipe matching is exposed through an API, and the Streamlit interface presents the result in a readable form. Four unit tests validate important matching behaviors. The score remains a heuristic and the application currently returns one best match without personalization.

7.2 Future Scope

- Evaluate ranking quality against a larger, manually labelled query set and tune the coverage and fuzzy thresholds from measured results.
- Improve ingredient parsing for synonyms, regional names, plural forms, transliterations, and multiword ingredients.
- Return several ranked recipe candidates and allow filters for cuisine, course, diet, and preparation time.
- Add automated API and browser-level integration tests.
- Add optional user accounts, saved ingredient lists, ratings, and personalized ranking if persistent storage is introduced.
- Improve accessibility and provide a user-controlled instruction reveal speed.
- Add data-update validation and a clear process for refreshing the recipe workbook.

A. API AND LOCAL EXECUTION

Run the backend from its `backend` folder and the frontend from its `frontend` folder:

```
python -m pip install -r requirements.txt
python -m uvicorn main:app --reload

streamlit run web_ui.py
```

Ingredient suggestions: `GET /ingredients`.
Recipe search: `POST /chat`.

A recipe search request has this form:

```
{
  "ingredients": ["onion", "tomato"]
}
```

The API documentation is available while the backend is running:

`http://127.0.0.1:8000/docs`

A.1 Project Limitations

The system depends on the provided workbook and does not verify that a recipe is safe for a particular allergy or dietary condition. Similarity scores are based on normalized text and should not be interpreted as nutritional or culinary guarantees. The current backend loads its dataset into memory at startup, and changes to the workbook require restarting the backend.

BIBLIOGRAPHY

[1] Francesco Ricci, Lior Rokach, and Bracha Shapira, editors, *Recommender Systems Handbook*, 2nd ed., Springer, 2015. doi: 10.1007/978-1-4899-7637-6.

[2] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008. https://nlp.stanford.edu/IR-book/.

[3] FastAPI, "FastAPI Documentation." https://fastapi.tiangolo.com/.

[4] Streamlit, "Streamlit Documentation." https://docs.streamlit.io/.

[5] RapidFuzz, "RapidFuzz Documentation." https://rapidfuzz.github.io/RapidFuzz/.

[6] pandas development team, "pandas Documentation." https://pandas.pydata.org/docs/.

[7] openpyxl, "openpyxl Documentation." https://openpyxl.readthedocs.io/.